Arithmetic Logic Unit (ALU) Design and Testing

Project Report

Author Name : Vipul Sharma
Employee ID : 6951

Project Title : Arithmetic Logic Unit (ALU) Design and Testing

May 2026

ABSTRACT

The Arithmetic Logic Unit (ALU) is an important part of digital systems and processors. It performs different arithmetic and logical operations required for data processing. In this project, a parameterized ALU was designed and verified using Verilog HDL. The ALU supports multiple arithmetic and logical operations such as addition, subtraction, comparison, shift, rotate, and signed operations. It also generates status outputs like carry, overflow, error, and comparison flags.

The ALU design was written as a synthesizable RTL module with configurable data width, making the design flexible and reusable for different applications. An 8-bit configuration was used for implementation and testing. A self-checking Verilog testbench was developed to verify the functionality of the ALU. The verification environment included randomized test cases, directed test scenarios, a reference model, and automatic result comparison.

Different operating conditions were tested including normal operations, boundary values, signed arithmetic, overflow cases, multiplication pipeline operations, rotate functions, invalid inputs, unknown values, and clock-enable conditions. Simulation and debugging were performed using QuestaSim.

During verification, a few functional issues were identified and corrected. These included missing comparison flag updates in signed arithmetic operations and result truncation problems in increment and decrement operations. After fixing the issues, the design achieved high functional accuracy and coverage results.

Contents

List of Figures

List of Tables

Chapter 1

INTRODUCTION

An Arithmetic Logic Unit (ALU) is one of the main building blocks of a digital system. It is used in processors, controllers, and embedded systems to perform arithmetic and logical calculations. The ALU is responsible for executing operations such as addition, subtraction, comparison, shifting, and bitwise logic functions.

In this project, a parameterized ALU was designed and verified using Verilog HDL. The design allows the operand size to be changed through parameters, which makes the ALU flexible and reusable for different applications. In this implementation, the default data width used is 8 bits.

The ALU is synchronous in nature and works with a clock signal. A Clock Enable (CE) signal is also provided to control when the outputs should update. The design supports several arithmetic and logical operations.

The arithmetic section includes operations like:

- Addition and subtraction
- Addition and subtraction with carry input
- Increment and decrement operations
- Signed addition and signed subtraction
- Comparison operation
- Pipelined multiplication operations
-

The logical section supports:

- AND, NAND, OR, NOR
- XOR and XNOR
- NOT operations
- Left and right shift operations
- Left and right rotate operations

The ALU also generates important status signals during operation:

- Carry Out (COUT)
- Overflow flag (OFLOW)
- Greater flag (G)
- Less flag (L)
- Equal flag (E)
- Error flag (ERR)

These flags help in checking arithmetic conditions, comparisons, and invalid input cases.

To verify the design, a self-checking Verilog testbench was developed. The testbench automatically compares the ALU outputs with expected results generated from a reference model. Different test cases such as normal operations, boundary conditions, invalid inputs, signed operations, and pipeline operations were simulated to ensure correct functionality of the design.

Chapter 2

OBJECTIVES

The main goals of this project are listed below:

- To understand the working and structure of a parameterized ALU
- To design and verify the ALU using Verilog HDL
- To create a self-checking testbench for automatic verification
- To verify all arithmetic and logical operations supported by the ALU
- To check the correctness of status flags such as COUT, OFLOW, G, L, E, and ERR
- To test signed arithmetic operations and verify overflow conditions
- To verify the functionality of pipelined multiplication operations
- To test rotate and shift operations for different input conditions
- To verify invalid input handling using different INP_VALID combinations
- To test the behavior of the Clock Enable (CE) signal
- To apply both randomized and directed test cases for better verification
- To verify corner cases such as all zeros, maximum values, minimum values, and equal operands

- To identify functional bugs in the design and correct them
- To perform simulation and coverage analysis using QuestaSim
- To achieve high functional and code coverage for the ALU design

The overall objective of this project is to ensure that the ALU works correctly under different operating conditions and produces reliable outputs for all supported operations.

Chapter 3

DESIGN ARCHITECTURE

3.1 Overview

The ALU designed in this project is a parameterized synchronous RTL module written in Verilog HDL. The design can perform different arithmetic and logical operations based on the control signals MODE and CMD. Since the ALU is parameterized, the data width can be changed easily according to system requirements. In this project, an 8-bit configuration is used.

The ALU operates with a clock signal and updates outputs only when the Clock Enable (CE) signal is active. Input values are registered on the positive edge of the clock, and operations are performed using these registered inputs.

The architecture of the ALU is divided into different functional blocks:

- MODE = 1 selects arithmetic operations
- MODE = 0 selects logical operations

Arithmetic Unit

- Addition
- Subtraction
- Addition with carry input
- Subtraction with carry input
- Increment and decrement operations
- Comparison operation

Signed Arithmetic Unit

- Overflow flag
- Greater than flag
- Less than flag
- Equal flag

Pipelined Multiplication Unit

The ALU includes two multiplication operations:

- MUL_INC : (OPA + 1) × (OPB + 1)
- MUL_SHL : (OPA << 1) × OPB

These operations are implemented using a 2-cycle pipeline.

Logical Unit

This block performs bitwise logical operations such as:

- AND
- NAND
- OR
- NOR
- XOR
- XNOR
- NOT operations

Flag Generation Logic

The ALU generates different status flags during execution:

- COUT (Carry Out)
- OFLOW (Overflow)
- G (Greater)
- L (Less)
- E (Equal)
- ERR (Error)

These flags help in checking the correctness and status of ALU operations.

3.2 ALU Block Diagram

Figure 3.1 presents the complete block diagram of the parameterized ALU architecture.

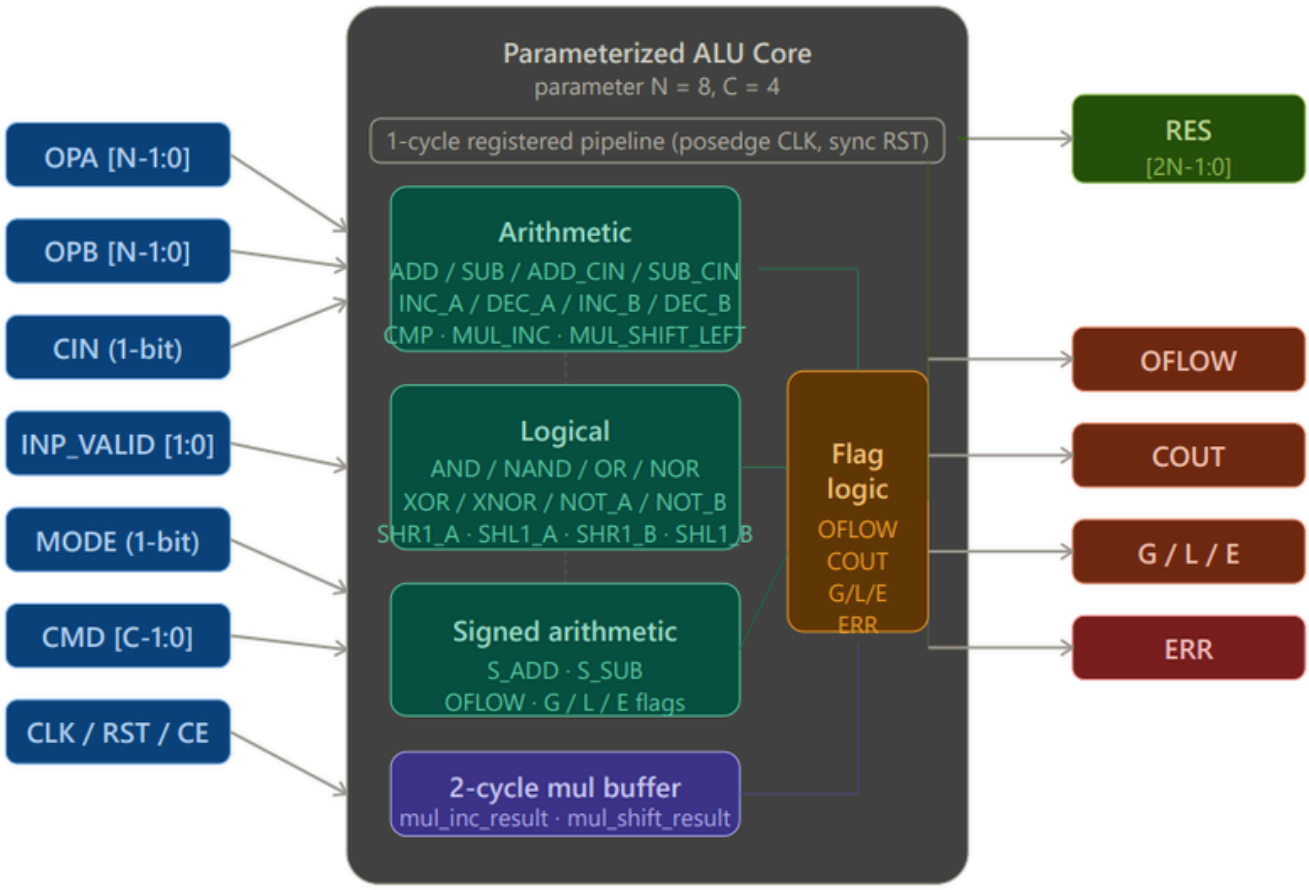

Figure 3.1: Parameterized ALU Architecture

Chapter 4

INPUT SIGNALS

Signal	Width	Description
OPA	N-bit (default 8-bit)	First operand input
OPB	N-bit (default 8-bit)	Second operand input
CIN	1-bit	Carry input used in ADD_CIN and SUB_CIN operations
CLK	1-bit	Positive edge triggered clock signal
RST	1-bit	Active-high reset signal used to clear outputs and flags
CE	1-bit	Clock enable signal; outputs update only when CE = 1
MODE	1-bit	Selects operation type: 1 = Arithmetic, 0 = Logical
INP_VALID	2-bit	Indicates valid operands: V_BOTH=11, V_A=01, V_B=10,
CMD	4-bit	Command signal used to select ALU operation

Chapter 5

OUTPUT SIGNALS

Signal	Width	Description
RES	2 × N (default 16-bit)	ALU operation resul
OFLOW	1-bit	Overflow flag generated during SUB, SUB_CIN, SADD, and SSUB operations.
COUT	1-bit	Carry-out flag generated for ADD, ADD_CIN, and signed arithmetic operations.
G	1-bit	Asserted when OPA > OPB
L	1-bit	Asserted when OPA < OPB
E	1-bit	Asserted when OPA == OPB
ERR	1-bit	Error flag asserted when INP_VALID is not valid for the selected operation.

Chapter 6

SUPPORTED OPERATIONS

The ALU supports both arithmetic and logical operations. The operation performed depends on the value of the MODE signal.

- MODE = 1 → Arithmetic Operations (13 operations)
- MODE = 0 → Logical Operations (13 operations)

6.1 Arithmetic Operations

(MODE = 1)

CMD	Mnemonic	Description
4'h0	ADD	RES = OPA + OPB; generates COUT; requires
4'h1	SUB	RES = {8'h00, OPA} - {8'h00, OPB}; OFLOW asserted if OPB > OPA; requires V_BOTH
4'h2	ADD_CIN	RES = OPA + OPB + CIN; generates COUT; requires V_BOTH
4'h3	SUB_CIN	RES = OPA - OPB - CIN; OFLOW asserted if OPA < OPB + CIN; requires

4'h4	INC_A	RES = {8'h00, OPA + 1}; requires V_A or V_BOTH
4'h5	DEC_A	RES = {8'h00, OPA - 1}; requires V_A or V_BOTH
4'h6	INC_B	RES = {8'h00, OPB + 1}; requires V_B or V_BOTH
4'h7	DEC_B	RES = {8'h00, OPB - 1}; requires V_B or V_BOTH
4'h8	CMP	RES = 0; comparator flags G, L, E generated using OPA and OPB; requires V_BOTH
4'h9	MUL_INC	RES = (OPA + 1) × (OPB + 1); 2-cycle pipelined operation; requires V_BOTH
4'hA	MUL_SHL	RES = (OPA << 1) × OPB; 2-cycle pipelined operation; requires V_BOTH
4'hB	SADD	Signed addition operation with G/L/E, OFLOW, and COUT flag generation; requires V_BOTH
4'hC	SSUB	Signed subtraction operation with G/L/E, OFLOW, and COUT flag generation; requires V_BOTH

6.2 Logical Operations (MODE = 0)

CMD	Mnemonic	Function
4'h0	AND	OPA & OPB
4'h1	NAND	~(OPA & OPB)
4'h2	OR	OPA | OPB
4'h3	NOR	~(OPA | OPB)
4'h4	XOR	OPA ^ OPB
4'h5	XNOR	~(OPA ^ OPB)
4'h6	NOT_A	~OPA
4'h7	NOT_B	~OPB
4'h8	SHR1_A	OPA >> 1
4'h9	SHL1_A	OPA << 1
4'hA	SHR1_B	OPB >> 1
4'hB	SHL1_B	OPB << 1
4'hC	ROL_A_B	Rotate OPA left by OPB bits
4'hD	ROR_A_B	Rotate OPA right by OPB bits

Chapter 7

TIMING BEHAVIOUR

TThe ALU works synchronously with the clock signal. Input data and control signals are sampled on the positive edge of the clock when the Clock Enable (CE) signal is active. The outputs are updated only when CE = 1; otherwise, the ALU holds the previous output values.

The operation flow of the ALU can be divided into three stages:

- Input sampling stage
- Operation execution stage
- Output update stage

The reset signal initializes the complete design. When RST is asserted:

- All output signals are cleared
- Status flags are reset to zero
- Internal registers are initialized to default values

The multiplication operations (MUL_INC and MUL_SHL) are implemented using a 2-cycle pipeline mechanism. Because of this, multiplication results appear after two clock cycles instead of one cycle.

7.1 Waveform Analysis

Simulation waveforms were analyzed to verify the correct behavior of the ALU under different operating conditions. The waveforms demonstrate:

- Arithmetic and logical operation execution
- Correct output generation for different CMD values
- Carry and overflow flag behavior
- Comparator flag generation (G, L, E)
- Pipeline delay in multiplication operations
- Rotate and shift functionality
- Reset and Clock Enable operation
- Error flag generation for invalid inputs

Figure 6.1: ALU Timing Waveform (ADD, SADD, SHL1 A Operations)

Chapter 8

CONCLUSION

The parameterized ALU was successfully designed and verified using Verilog HDL and 77QuestaSim. All 26 operations were tested using a self-checking testbench, including arithmetic, logical, signed, shift, rotate, and pipelined multiplication operations.

The ALU correctly generated status flags such as COUT, OFLOW, G, L, E, and ERR, while also supporting Clock Enable and asynchronous Reset functionality.

total code coverage of 77.90% coverage was achieved.